

MLSys 2026 Google Competition 참가 후기

데이터플랫폼팀 병장 손량

June 28, 2026

Outline

TL;DR

Background

- Systems View on Deep Learning Models
- Computation Graphs
- Hardware for Deep Learning
- Operator Fusion

Solving the Problem

- Problem Analysis
- Global Optimization
- Local Optimization

Meta-strategy

Outline

TL;DR

Background

- Systems View on Deep Learning Models
- Computation Graphs
- Hardware for Deep Learning
- Operator Fusion

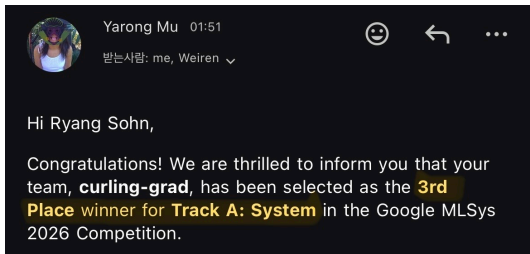
Solving the Problem

- Problem Analysis
- Global Optimization
- Local Optimization

Meta-strategy

- MLSys 학회의 대회 트랙에서 구글이 개최한 그래프 스케줄링 대회에 참가했습니다.
 - MLSys는 machine learning system을 주제로 하는 국제 학회입니다.
 - 목표: 계산 그래프의 최적화 알고리즘을 설계하여 NPU에서 최적의 성능을 달성합니다.
 - 최근 중요도가 높은 머신러닝 시스템의 최적화와 밀접한 관련이 있습니다.

- MLSys 학회의 대회 트랙에서 구글이 개최한 그래프 스케줄링 대회에 참가했습니다.
 - MLSys는 machine learning system을 주제로 하는 국제 학회입니다.
 - 목표: 계산 그래프의 최적화 알고리즘을 설계하여 NPU에서 최적의 성능을 달성합니다.
 - 최근 중요도가 높은 머신러닝 시스템의 최적화와 밀접한 관련이 있습니다.
- 2개월 동안 열심히 연구하고 구현한 끝에 **최종 3등**을 달성했습니다.



Outline

TL;DR

Background

- Systems View on Deep Learning Models
- Computation Graphs
- Hardware for Deep Learning
- Operator Fusion

Solving the Problem

- Problem Analysis
- Global Optimization
- Local Optimization

Meta-strategy

Systems View on Deep Learning Models

딥러닝 모델도 프로그램이다

- CNN, RNN, Transformer 등 딥러닝 모델의 구조는 다양한 종류가 있습니다.
- 딥러닝 모델은 다양한 구조를 가지지만, 중요한 특성을 공유합니다.
 - 텐서 연산으로 이루어져 있습니다.
 - 전체적인 control flow가 대부분 한 방향만을 따라갑니다.

Systems View on Deep Learning Models

딥러닝 모델도 프로그램이다

- CNN, RNN, Transformer 등 딥러닝 모델의 구조는 다양한 종류가 있습니다.
- 딥러닝 모델은 다양한 구조를 가지지만, 중요한 특성을 공유합니다.
 - 텐서 연산으로 이루어져 있습니다.
 - 전체적인 control flow가 대부분 한 방향만을 따라갑니다.
- 텐서: $T: \underbrace{V^* \times \dots \times V^*}_{p \text{ times}} \times \underbrace{V \times \dots \times V}_{q \text{ times}} \rightarrow \mathbb{R}$ where V is a vector space ...

Systems View on Deep Learning Models

딥러닝 모델도 프로그램이다

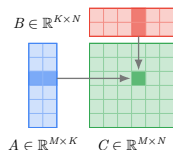
- CNN, RNN, Transformer 등 딥러닝 모델의 구조는 다양한 종류가 있습니다.
- 딥러닝 모델은 다양한 구조를 가지지만, 중요한 특성을 공유합니다.
 - 텐서 연산으로 이루어져 있습니다.
 - 전체적인 control flow가 대부분 한 방향만을 따라갑니다.
- 텐서: 다차원 배열

Systems View on Deep Learning Models

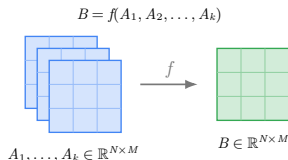
딥러닝 모델도 프로그램이다

- CNN, RNN, Transformer 등 딥러닝 모델의 구조는 다양한 종류가 있습니다.
- 딥러닝 모델은 다양한 구조를 가지지만, 중요한 특성을 공유합니다.
 - **텐서 연산**으로 이루어져 있습니다.
 - 전체적인 control flow가 대부분 한 방향만을 따라갑니다.
- **텐서**: 다차원 배열

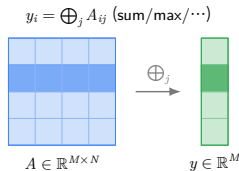
Matrix Multiplication



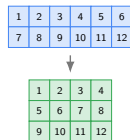
Elementwise



Reduction



Reshape
same data, new shape



Computation Graphs

Example: Gemma 4 Decoder Block

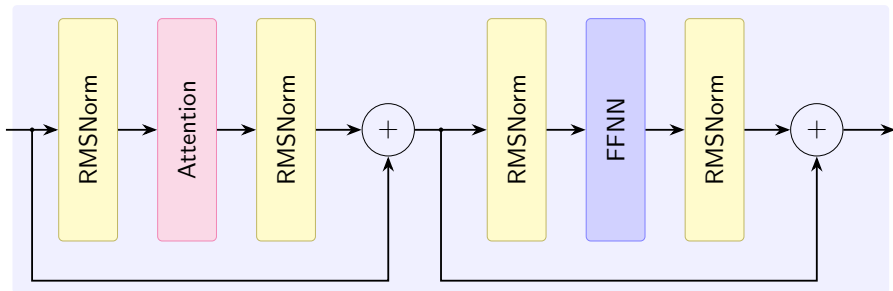


Figure: Gemma 4 Decoder Block

Computation Graphs

Example: Gemma 4 Decoder Block

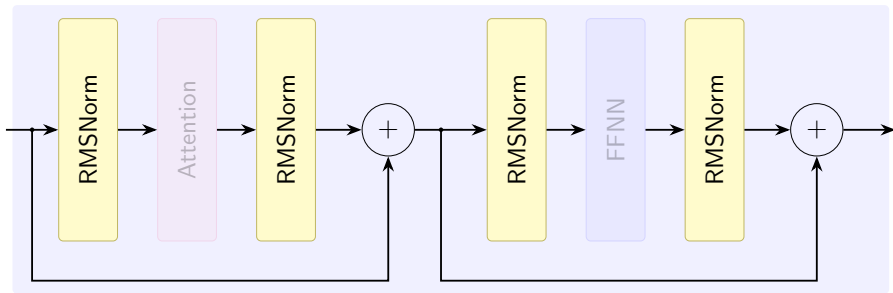


Figure: Gemma 4 Decoder Block

Computation Graphs

RMSNorm

RMSNorm의 정의는 다음과 같습니다.

$$[\text{RMSNorm}(x)]_i = \frac{x_i w_i}{\sqrt{\text{mean}(x^2) + \epsilon}}$$

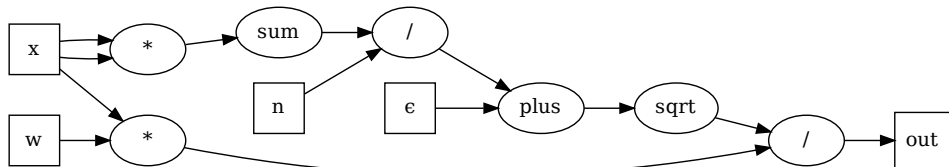
Computation Graphs

RMSNorm

RMSNorm의 정의는 다음과 같습니다.

$$[\text{RMSNorm}(x)]_i = \frac{x_i w_i}{\sqrt{\text{mean}(x^2) + \epsilon}}$$

입력 텐서는 x , w , ϵ , n 이고, 텐서 연산자와 입력, 출력은 다음과 같이 연결됩니다.



Computation Graphs

Example: Gemma 4 Decoder Block

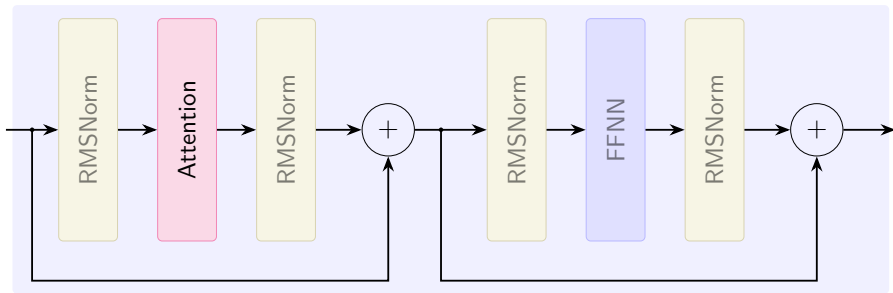


Figure: Gemma 4 Decoder Block

Computation Graphs

Attention

Attention의 정의는 다음과 같습니다.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V, \quad [\text{softmax}(x)]_i = \frac{\exp(x_i)}{\sum_i \exp(x_i)}$$

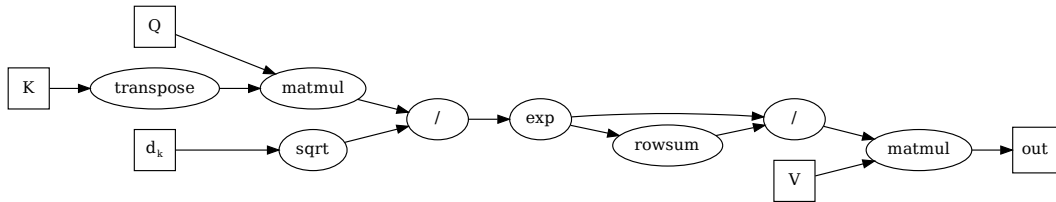
Computation Graphs

Attention

Attention의 정의는 다음과 같습니다.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V, \quad [\text{softmax}(x)]_i = \frac{\exp(x_i)}{\sum_i \exp(x_i)}$$

입력 텐서는 Q, K, V, d_k 이고, 계산 그래프는 다음과 같습니다.



Hardware for Deep Learning

Tiled Matmul

- 딥러닝에서 가장 많이 사용되는 연산은 행렬 곱셈입니다.
 - 행렬 곱셈을 빠르게 할수록 딥러닝 모델의 실행 속도를 개선할 수 있습니다.
- 이렇게 행렬을 타일 단위로 곱하면 여러 이점이 있습니다.
 1. 같은 연산을 위해 사용해야 하는 명령어 수를 줄입니다.

Hardware for Deep Learning

Tiled Matmul

- 딥러닝에서 가장 많이 사용되는 연산은 행렬 곱셈입니다.
 - 행렬 곱셈을 빠르게 할수록 딥러닝 모델의 실행 속도를 개선할 수 있습니다.
- 이렇게 행렬을 타일 단위로 곱하면 여러 이점이 있습니다.
 1. 같은 연산을 위해 사용해야 하는 명령어 수를 줄입니다.
 2. 메모리 접근 수를 줄입니다.

Hardware for Deep Learning

Tiled Matmul

- 딥러닝에서 가장 많이 사용되는 연산은 행렬 곱셈입니다.
 - 행렬 곱셈을 빠르게 할수록 딥러닝 모델의 실행 속도를 개선할 수 있습니다.
- 이렇게 행렬을 타일 단위로 곱하면 여러 이점이 있습니다.
 1. 같은 연산을 위해 사용해야 하는 명령어 수를 줄입니다.
 2. 메모리 접근 수를 줄입니다.

a_{11}	a_{12}	a_{13}	a_{14}
a_{21}	a_{22}	a_{23}	a_{24}
a_{31}	a_{32}	a_{33}	a_{34}
a_{41}	a_{42}	a_{43}	a_{44}

 $\times$

b_{11}	b_{12}	b_{13}	b_{14}
b_{21}	b_{22}	b_{23}	b_{24}
b_{31}	b_{32}	b_{33}	b_{34}
b_{41}	b_{42}	b_{43}	b_{44}

 $=$

c_{11}	c_{12}	c_{13}	c_{14}
c_{21}	c_{22}	c_{23}	c_{24}
c_{31}	c_{32}	c_{33}	c_{34}
c_{41}	c_{42}	c_{43}	c_{44}

Figure: 1x1 원소를 계산하는데 원소 8개, 총 $8 \times 16 = 128$ 개를 읽어야 합니다.

Hardware for Deep Learning

Tiled Matmul

- 딥러닝에서 가장 많이 사용되는 연산은 행렬 곱셈입니다.
 - 행렬 곱셈을 빠르게 할수록 딥러닝 모델의 실행 속도를 개선할 수 있습니다.
- 이렇게 행렬을 타일 단위로 곱하면 여러 이점이 있습니다.
 1. 같은 연산을 위해 사용해야 하는 명령어 수를 줄입니다.
 2. 메모리 접근 수를 줄입니다.

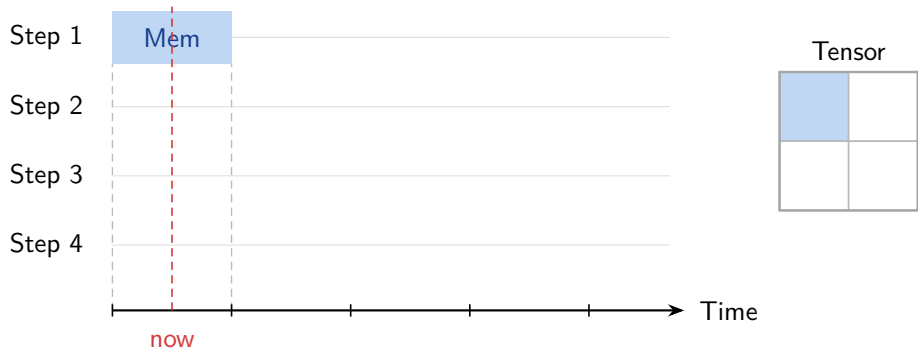
$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} & c_{14} \\ c_{21} & c_{22} & c_{23} & c_{24} \\ c_{31} & c_{32} & c_{33} & c_{34} \\ c_{41} & c_{42} & c_{43} & c_{44} \end{bmatrix}$$

Figure: 2x2 원소를 계산하는데 원소 16개, 총 $16 \times 4 = 64$ 개를 읽어야 합니다.

Hardware for Deep Learning

Pipelining

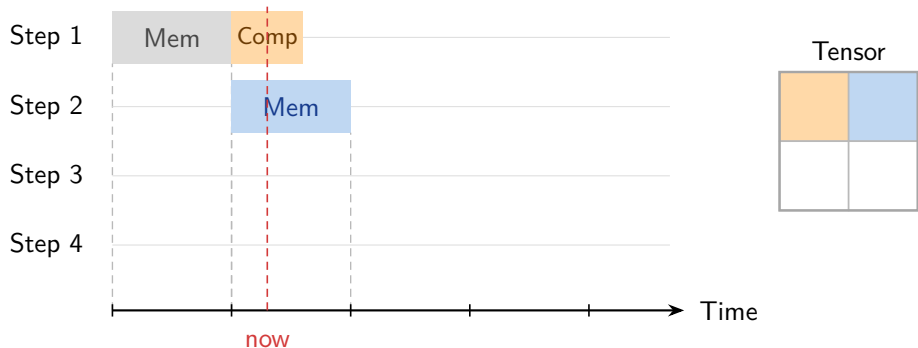
일반적으로 텐서 연산은 메모리 접근과 연산을 동시에 수행할 수 있습니다.



Hardware for Deep Learning

Pipelining

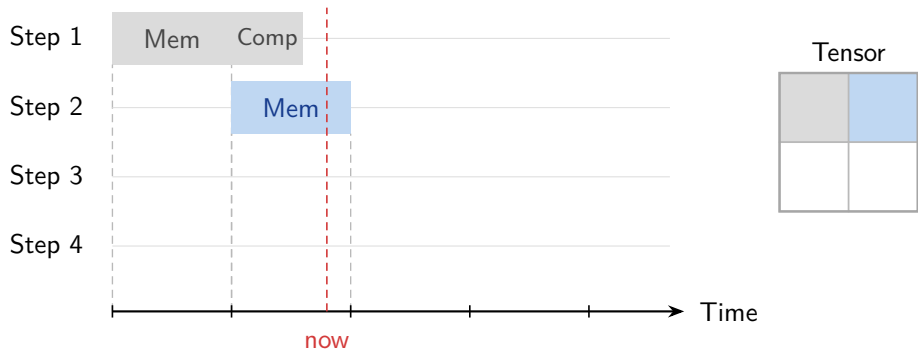
일반적으로 텐서 연산은 메모리 접근과 연산을 동시에 수행할 수 있습니다.



Hardware for Deep Learning

Pipelining

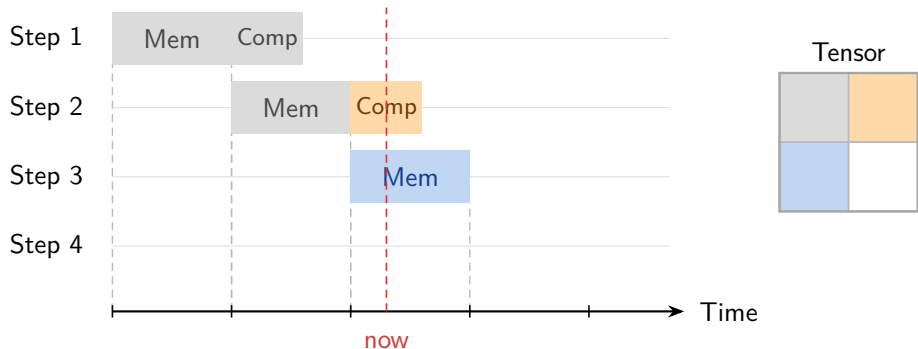
일반적으로 텐서 연산은 메모리 접근과 연산을 동시에 수행할 수 있습니다.



Hardware for Deep Learning

Pipelining

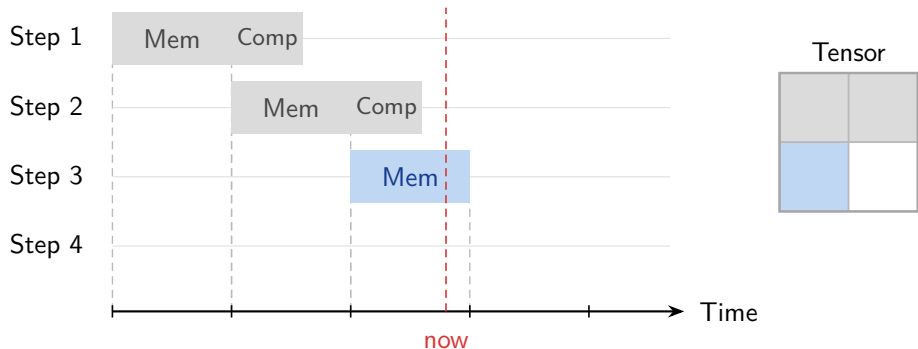
일반적으로 텐서 연산은 메모리 접근과 연산을 동시에 수행할 수 있습니다.



Hardware for Deep Learning

Pipelining

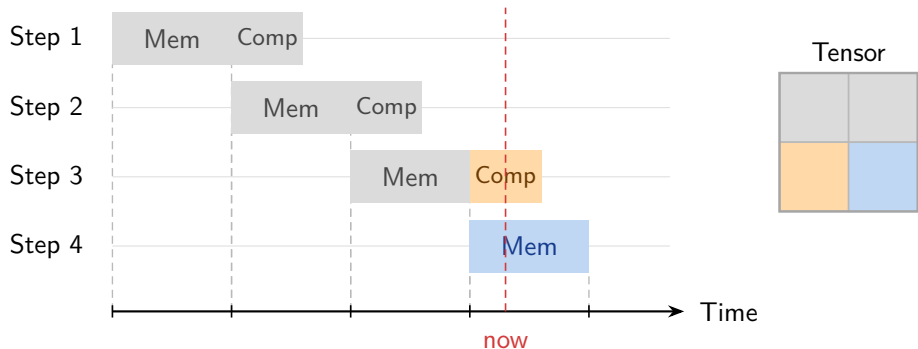
일반적으로 텐서 연산은 메모리 접근과 연산을 동시에 수행할 수 있습니다.



Hardware for Deep Learning

Pipelining

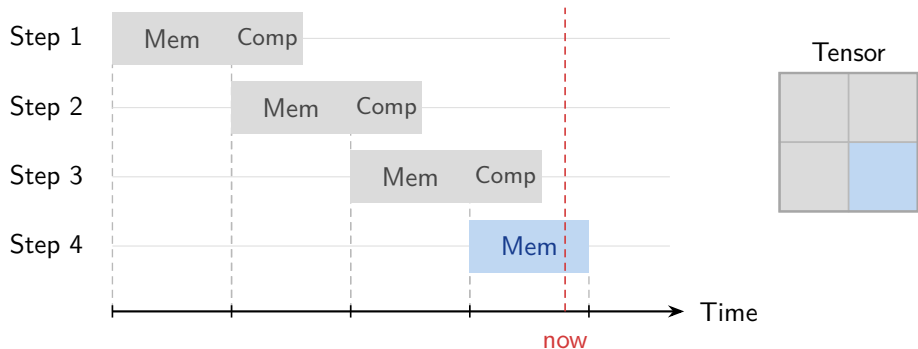
일반적으로 텐서 연산은 메모리 접근과 연산을 동시에 수행할 수 있습니다.



Hardware for Deep Learning

Pipelining

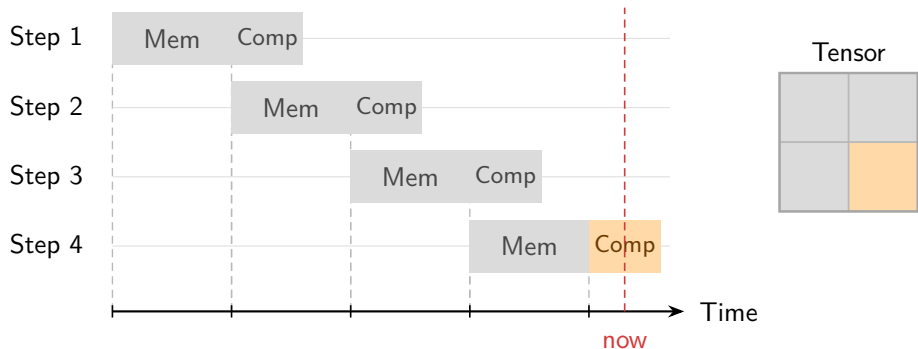
일반적으로 텐서 연산은 메모리 접근과 연산을 동시에 수행할 수 있습니다.



Hardware for Deep Learning

Pipelining

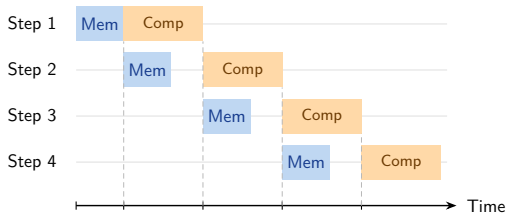
일반적으로 텐서 연산은 메모리 접근과 연산을 동시에 수행할 수 있습니다.



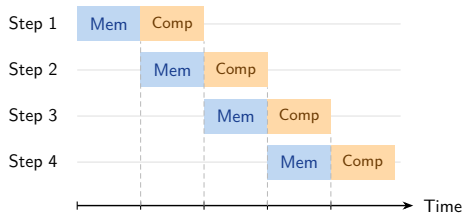
Hardware for Deep Learning

Roofline Model

- 메모리 접근과 연산을 병렬화하면 전체 속도는 둘 중 느린 것에 의해 정해집니다.
- 대부분의 하드웨어에서 메모리는 연산 속도보다 느립니다.
 - B200 GPU: 2,250 TFLOPS (BF16) vs 8 TB/s
 - 연산 속도는 더 큰 칩을 만들면 늘릴 수 있지만, 메모리는 빠르게 하기 어렵습니다.
 - 보통 compute bound가 되는 것을 목표로 최적화합니다.



(a) Compute bound



(b) Balanced

Operator Fusion

Example: Chain of Elementwise Operators

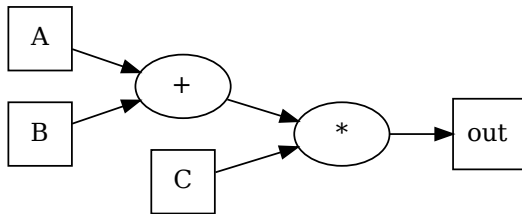
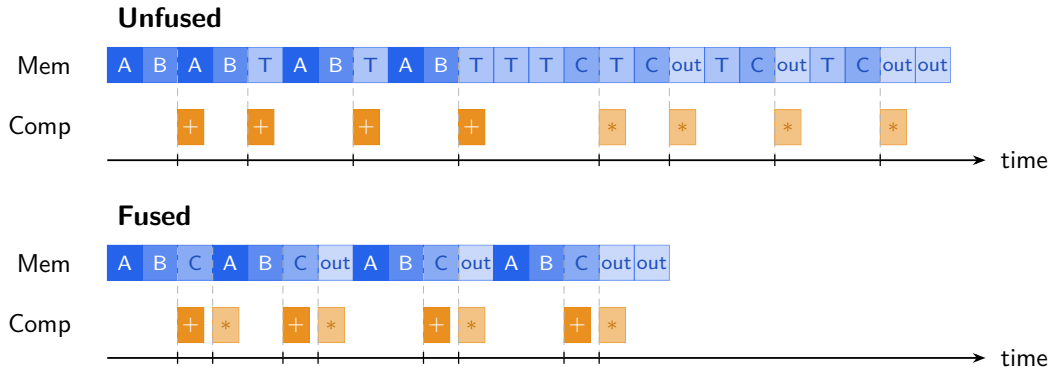


Figure: $(A + B) * C$ 의 계산 그래프

- 덧셈과 곱셈을 따로 실행한다고 가정합니다.
 - 덧셈 결과를 메모리에 저장하고 곱셈하기 전에 다시 읽어야 합니다.
 - 덧셈과 곱셈 연산보다 메모리 접근에 시간이 낭비됩니다.
- 덧셈과 곱셈을 한번에 실행하면 낭비되는 시간을 줄일 수 있습니다.

Operator Fusion

Example: Chain of Elementwise Operators



여전히 memory bound 이지만, 실행 시간이 훨씬 줄었습니다.

Operator Fusion

계산을 더 많이 하고도 빨라질 수 있다?

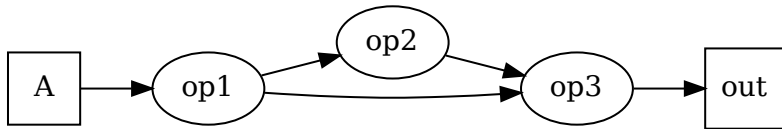


Figure: Residual connection이 있는 그래프

- op2와 op3를 한번에 계산할 수 없다고 가정합니다.
- op1과 op2를 합치면 op3에서 입력을 받을 수 없고, 반대도 마찬가지입니다.

Operator Fusion

계산을 더 많이 하고도 빨라질 수 있다?

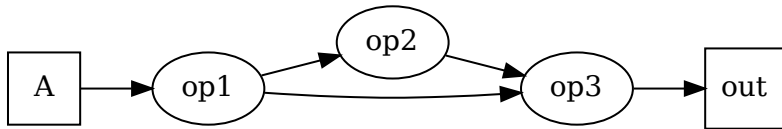


Figure: Residual connection이 있는 그래프

- op2와 op3를 한번에 계산할 수 없다고 가정합니다.
- op1과 op2를 합치면 op3에서 입력을 받을 수 없고, 반대도 마찬가지입니다.
- op1의 계산량이 적다면, (op1, op2), (op1, op3)로 합치는 것이 빠를 수 있습니다.
- FlashAttention과 같은 고성능 커널 설계에 이 아이디어가 이용된 바 있습니다.

Outline

TL;DR

Background

- Systems View on Deep Learning Models

- Computation Graphs

- Hardware for Deep Learning

- Operator Fusion

Solving the Problem

- Problem Analysis

- Global Optimization

- Local Optimization

Meta-strategy

Problem Analysis

문제의 목표

계산 그래프가 주어질 때, 하드웨어에서 빠르게 실행하는 전략을 찾아야 합니다.

Problem Analysis

문제의 목표

계산 그래프가 주어질 때, 하드웨어에서 빠르게 실행하는 전략을 찾아야 합니다.

- 문제에서 가정하는 하드웨어는 일반적인 NPU와 비슷한 구조를 갖고 있습니다.
 - 빠르고 공간이 제한된 메모리와 느리고 공간이 큰 두 종류의 메모리가 있습니다.
 - 계산 그래프를 노드 그룹 단위로, 텐서 연산을 타일 단위로 실행합니다.
 - 텐서 연산을 위해서는 입력 텐서의 타일을 빠른 메모리로 로드해야 합니다.
 - 메모리 접근과 계산을 동시에 수행할 수 있습니다.

Problem Analysis

문제의 목표

계산 그래프가 주어질 때, 하드웨어에서 빠르게 실행하는 전략을 찾아야 합니다.

- 문제에서 가정하는 하드웨어는 일반적인 NPU와 비슷한 구조를 갖고 있습니다.
 - 빠르고 공간이 제한된 메모리와 느리고 공간이 큰 두 종류의 메모리가 있습니다.
 - 계산 그래프를 노드 그룹 단위로, 텐서 연산을 타일 단위로 실행합니다.
 - 텐서 연산을 위해서는 입력 텐서의 타일을 빠른 메모리로 로드해야 합니다.
 - 메모리 접근과 계산을 동시에 수행할 수 있습니다.
- 입력으로 어떤 계산 그래프가 주어질지 알 수 없습니다.
 - 특정한 경우에 대해서만 실행 전략을 찾는 것으로는 부족합니다.
 - 가능한 입력에 대해서 일반적으로 빠른 실행 전략을 찾는 알고리즘을 설계해야 합니다.

Problem Analysis

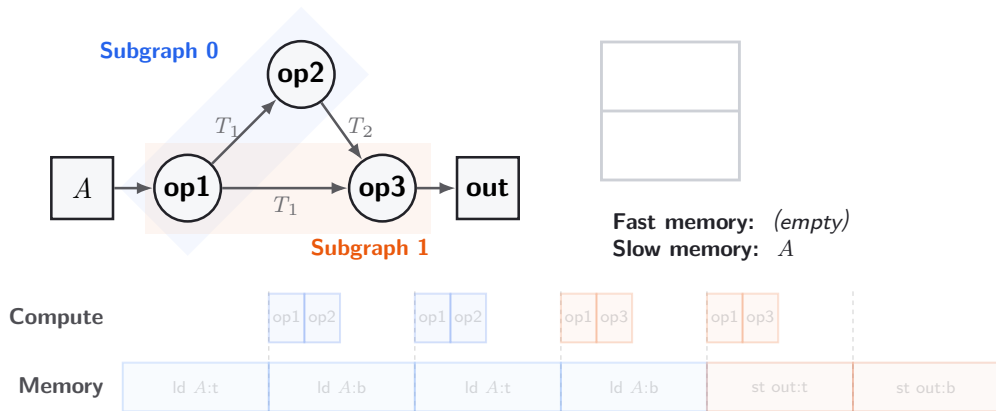
Execution Strategy

알고리즘은 크게 세 가지 변수에 대한 최적값을 찾아야 합니다.

1. 하드웨어에서 한번에 실행할 노드 그룹
2. 각 그룹에 있는 텐서 연산의 실행 타일 크기
3. 그룹의 출력 중, 느린 메모리를 거치지 않을 텐서

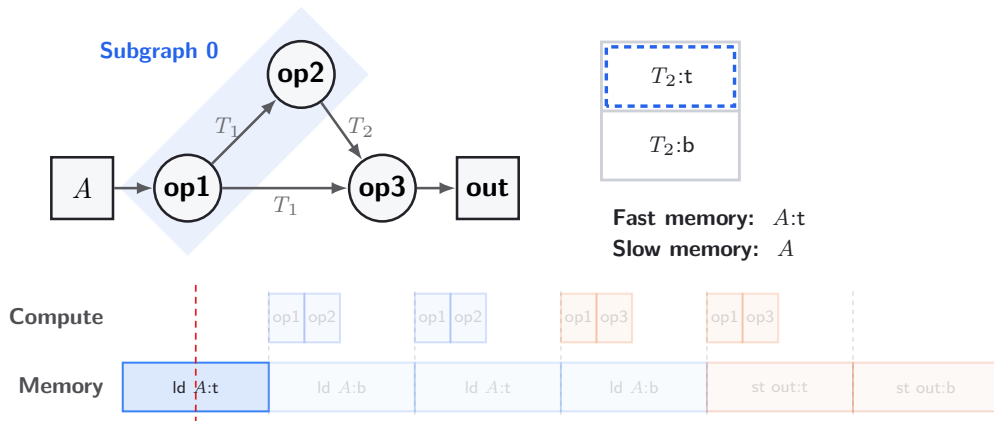
Problem Analysis

Example: Strategy on Residual Connection



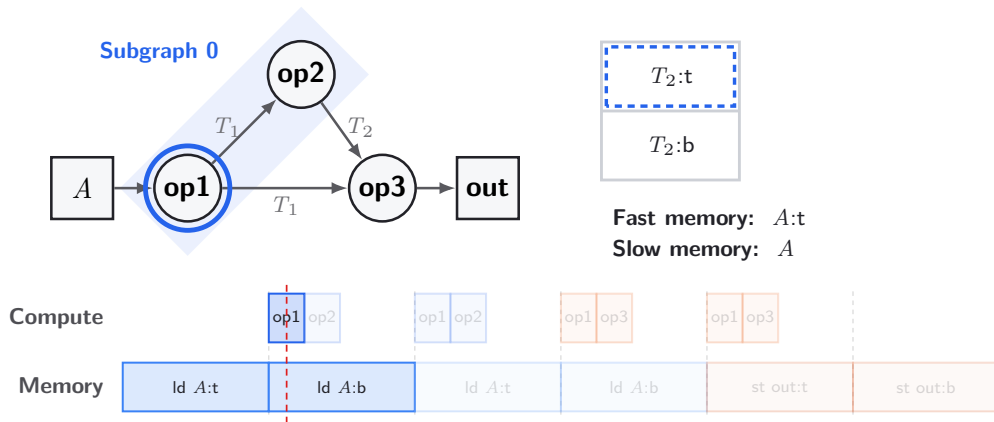
Problem Analysis

Example: Strategy on Residual Connection



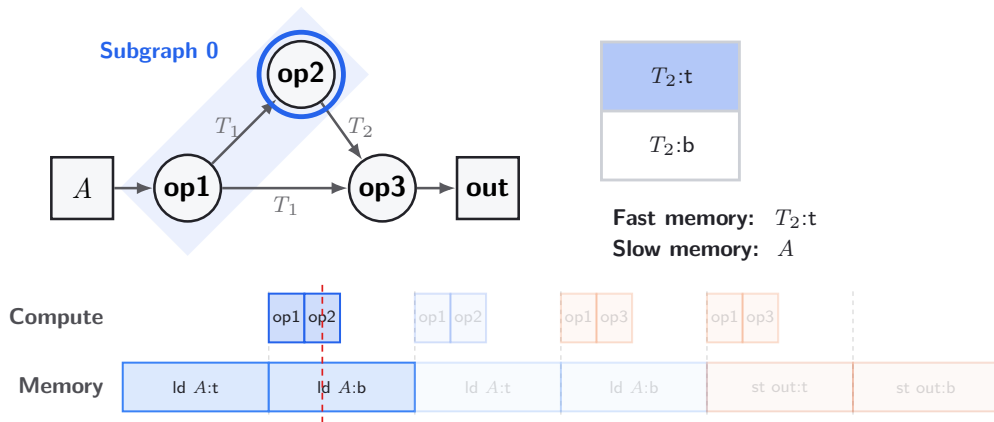
Problem Analysis

Example: Strategy on Residual Connection



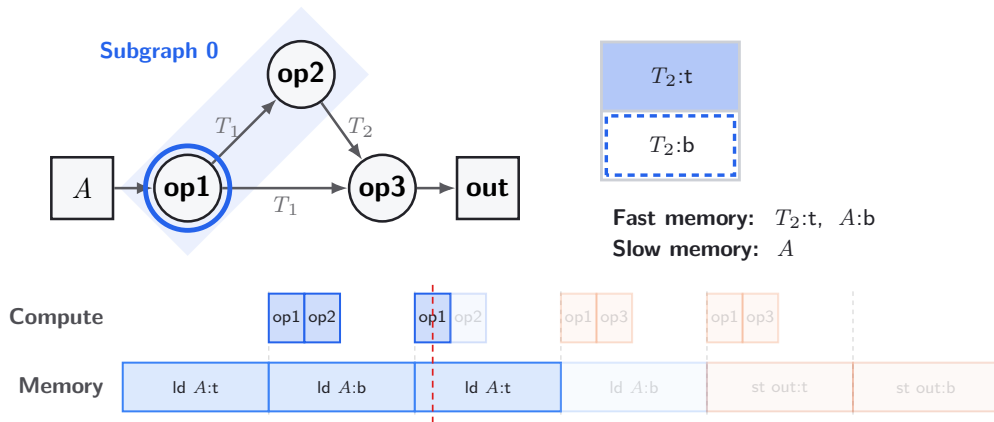
Problem Analysis

Example: Strategy on Residual Connection



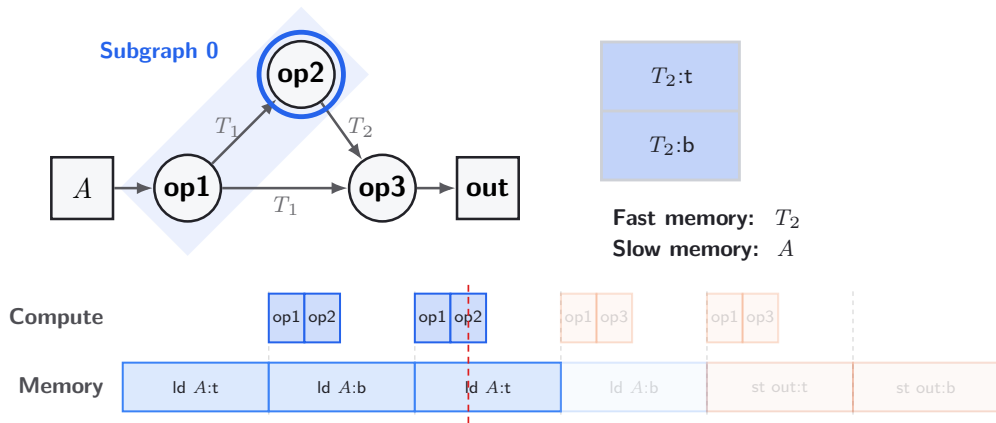
Problem Analysis

Example: Strategy on Residual Connection



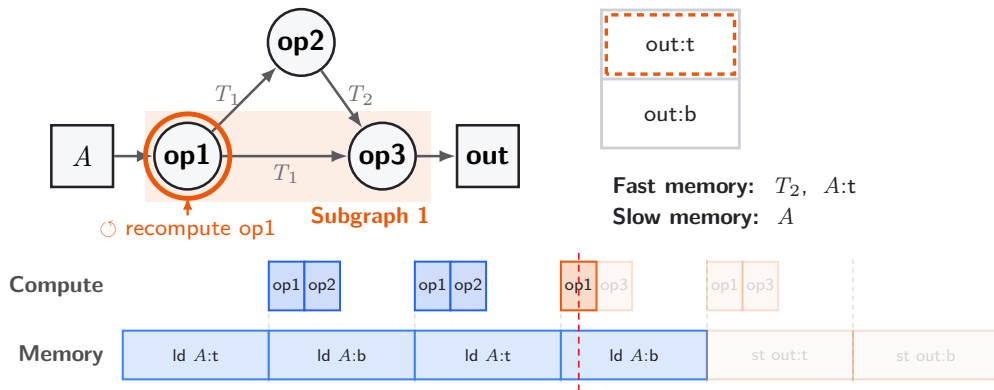
Problem Analysis

Example: Strategy on Residual Connection



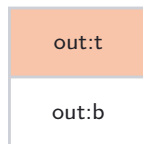
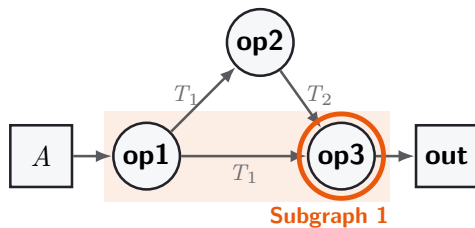
Problem Analysis

Example: Strategy on Residual Connection



Problem Analysis

Example: Strategy on Residual Connection

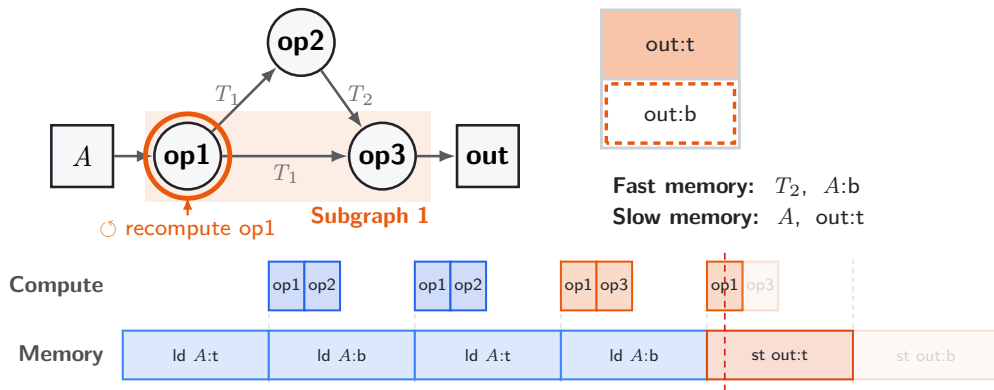


Fast memory: T_2 , $out:t$
Slow memory: A



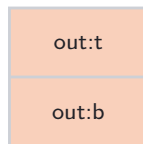
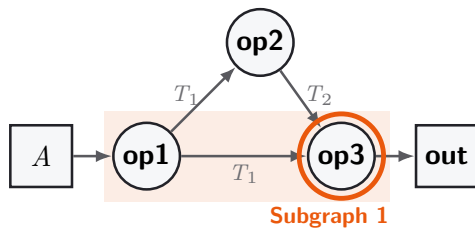
Problem Analysis

Example: Strategy on Residual Connection



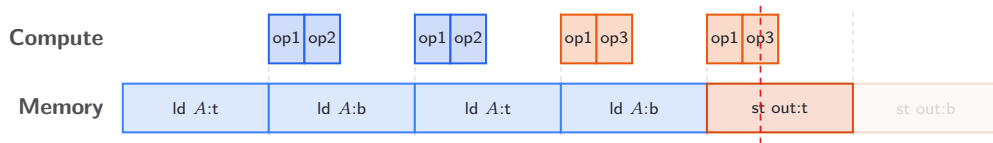
Problem Analysis

Example: Strategy on Residual Connection



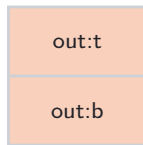
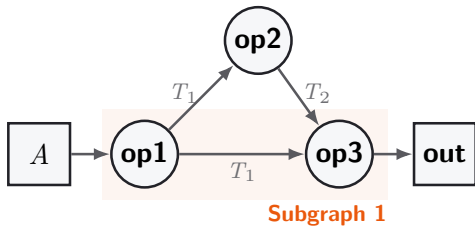
Fast memory: T_2 , out:b

Slow memory: A , out:t



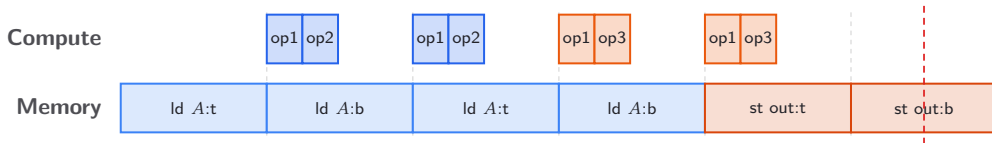
Problem Analysis

Example: Strategy on Residual Connection



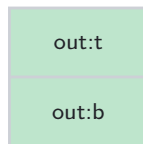
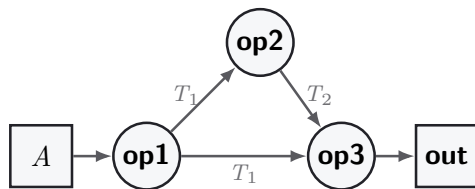
Fast memory: $out:b$

Slow memory: $A, out:t$

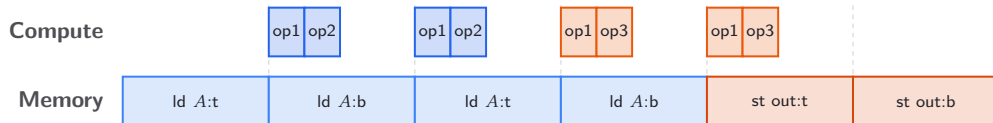


Problem Analysis

Example: Strategy on Residual Connection



Fast memory: *(empty)*
Slow memory: A , out



Problem Analysis

이거 웰노운 아닌가?

- 결국 문제의 핵심은 operator fusion을 잘하는 것입니다.
 - Operator fusion은 머신러닝 시스템 분야에서 활발히 연구되고 있는 주제입니다.
 - PyTorch, JAX, TensorRT 등 다양한 런타임에 구현되어 있습니다.
 - “코딩할 시간도 얼마 없는데 이미 잘 구현된 아이디어를 참고하면 되지 않을까?”

Problem Analysis

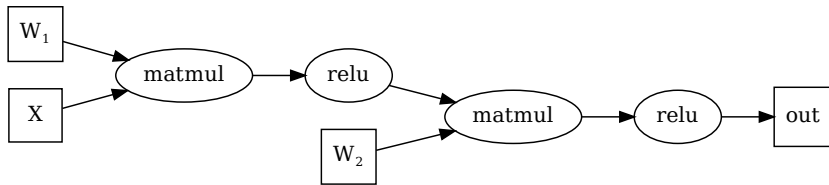
이거 웰노운 아닌가?

- 결국 문제의 핵심은 operator fusion을 잘하는 것입니다.
 - Operator fusion은 머신러닝 시스템 분야에서 활발히 연구되고 있는 주제입니다.
 - PyTorch, JAX, TensorRT 등 다양한 런타임에 구현되어 있습니다.
 - “코딩할 시간도 얼마 없는데 이미 잘 구현된 아이디어를 참고하면 되지 않을까?”
- 대부분의 런타임에서 operator fusion은 rule based 기법으로 구현합니다.
 - 자주 나타나는 패턴과 최적화 방식을 미리 입력해 두고, 패턴 매칭을 통해 최적화합니다.
 - 예시: 행렬 곱의 결과를 ReLU가 사용하는 경우, 두 연산을 함께 실행합니다.

Problem Analysis

이거 웰노운 아닌가?

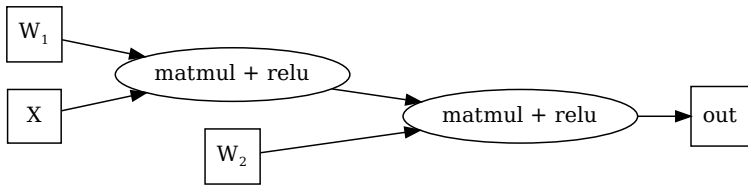
- 결국 문제의 핵심은 operator fusion을 잘하는 것입니다.
 - Operator fusion은 머신러닝 시스템 분야에서 활발히 연구되고 있는 주제입니다.
 - PyTorch, JAX, TensorRT 등 다양한 런타임에 구현되어 있습니다.
 - “코딩할 시간도 얼마 없는데 이미 잘 구현된 아이디어를 참고하면 되지 않을까?”
- 대부분의 런타임에서 operator fusion은 rule based 기법으로 구현합니다.
 - 자주 나타나는 패턴과 최적화 방식을 미리 입력해 두고, 패턴 매칭을 통해 최적화합니다.
 - 예시: 행렬 곱의 결과를 ReLU가 사용하는 경우, 두 연산을 함께 실행합니다.



Problem Analysis

이거 웰노운 아닌가?

- 결국 문제의 핵심은 operator fusion을 잘하는 것입니다.
 - Operator fusion은 머신러닝 시스템 분야에서 활발히 연구되고 있는 주제입니다.
 - PyTorch, JAX, TensorRT 등 다양한 런타임에 구현되어 있습니다.
 - “코딩할 시간도 얼마 없는데 이미 잘 구현된 아이디어를 참고하면 되지 않을까?”
- 대부분의 런타임에서 operator fusion은 rule based 기법으로 구현합니다.
 - 자주 나타나는 패턴과 최적화 방식을 미리 입력해 두고, 패턴 매칭을 통해 최적화합니다.
 - 예시: 행렬 곱의 결과를 ReLU가 사용하는 경우, 두 연산을 함께 실행합니다.



Problem Analysis

날로 먹기란 쉽지 않다

- Rule based 기법을 적용하려면 패턴을 그래프에서 찾기 쉬워야 합니다.
 - 일반적인 그래프는 행렬 곱, ReLU, reshape 등 다양한 종류의 연산이 있습니다.

Problem Analysis

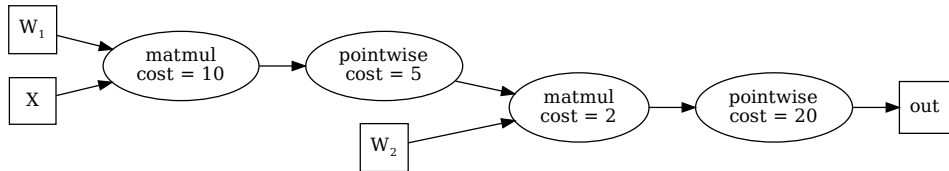
날로 먹기란 쉽지 않다

- Rule based 기법을 적용하려면 패턴을 그래프에서 찾기 쉬워야 합니다.
 - 일반적인 그래프는 행렬 곱, ReLU, reshape 등 다양한 종류의 연산이 있습니다.
 - 하지만 이 문제의 그래프에서는 연산 종류가 matmul, elementwise 두 종류뿐입니다.
 - 원래는 연산 종류로 실행 시간을 예상할 수 있지만, 문제에서는 임의로 주어집니다.

Problem Analysis

날로 먹기란 쉽지 않다

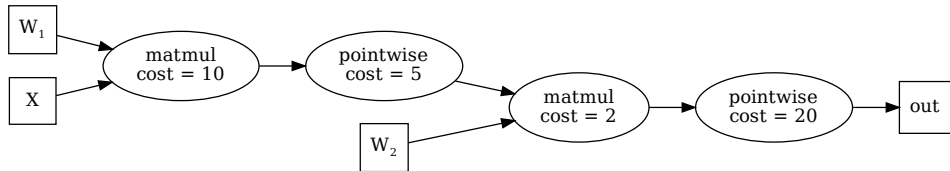
- Rule based 기법을 적용하려면 패턴을 그래프에서 찾기 쉬워야 합니다.
 - 일반적인 그래프는 행렬 곱, ReLU, reshape 등 다양한 종류의 연산이 있습니다.
 - 하지만 이 문제의 그래프에서는 연산 종류가 matmul, elementwise 두 종류뿐입니다.
 - 원래는 연산 종류로 실행 시간을 예상할 수 있지만, 문제에서는 임의로 주어집니다.



Problem Analysis

날로 먹기란 쉽지 않다

- Rule based 기법을 적용하려면 패턴을 그래프에서 찾기 쉬워야 합니다.
 - 일반적인 그래프는 행렬 곱, ReLU, reshape 등 다양한 종류의 연산이 있습니다.
 - 하지만 이 문제의 그래프에서는 연산 종류가 matmul, elementwise 두 종류뿐입니다.
 - 원래는 연산 종류로 실행 시간을 예상할 수 있지만, 문제에서는 임의로 주어집니다.



전체적인 전략

패턴 매칭 대신, 다양한 최적화 전략을 탐색하는 알고리즘을 구현해야 합니다.

Global Optimization

Defining the Search Space

- Operator fusion 탐색 공간은 매우 방대합니다.
 - 탐색 공간이 계산 그래프의 연산자 개수에 대해 지수적으로 늘어납니다.
 - “무식하게” 탐색하면 전역하기 전까지 답을 못 구할수도 있습니다.

Global Optimization

Defining the Search Space

- Operator fusion 탐색 공간은 매우 방대합니다.
 - 탐색 공간이 계산 그래프의 연산자 개수에 대해 지수적으로 늘어납니다.
 - “무식하게” 탐색하면 지구가 멸망하기 전까지 답을 못 구할수도 있습니다.

탐색 공간의 정의

효율적으로 탐색하되 최적에 가까운 답을 구할 수 있도록 공간을 정의해야 합니다.

Global Optimization

Defining the Search Space

- Operator fusion 탐색 공간은 매우 방대합니다.
 - 탐색 공간이 계산 그래프의 연산자 개수에 대해 지수적으로 늘어납니다.
 - “무식하게” 탐색하면 지구가 멸망하기 전까지 답을 못 구할수도 있습니다.

탐색 공간의 정의

효율적으로 탐색하되 최적에 가까운 답을 구할 수 있도록 공간을 정의해야 합니다.

- 탐색 공간 정의도 operator fusion에서 중요하기 때문에 다양한 연구가 있습니다.
- Optimal Kernel Orchestration for Tensor Programs with Korch (ASPLOS '24)
 - 간단한 operator fusion 외의 다양한 패턴도 효율적으로 탐색할 수 있습니다.

Global Optimization

Taming the Search Space

- N 개 연산자가 있을 때 동시에 실행할 수 있는 그룹은 2^N 개입니다.
 - 각 연산자마다 동시에 실행할 그룹에 있음/없음 2가지 경우가 가능하기 때문입니다.

Global Optimization

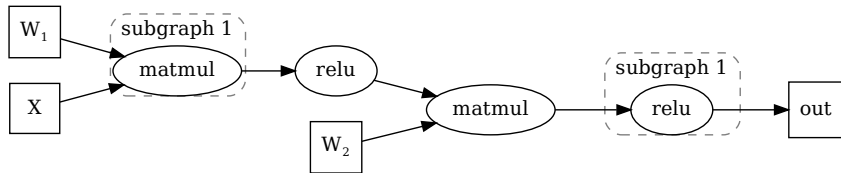
Taming the Search Space

- N 개 연산자가 있을 때 동시에 실행할 수 있는 그룹은 2^N 개입니다.
 - 각 연산자마다 동시에 실행할 그룹에 있음/없음 2가지 경우가 가능하기 때문입니다.
- 이 2^N 개 모두를 고려할 필요는 없다는 사실에 주목해야 합니다.

Global Optimization

Taming the Search Space

- N 개 연산자가 있을 때 동시에 실행할 수 있는 그룹은 2^N 개입니다.
 - 각 연산자마다 동시에 실행할 그룹에 있음/없음 2가지 경우가 가능하기 때문입니다.
- 이 2^N 개 모두를 고려할 필요는 없다는 사실에 주목해야 합니다.



- Korch에서는 의미 있는 경우만 탐색하기 위해 convex subgraph를 사용합니다.
 - Convex subgraph만을 고려하면 개수가 2^N 개에서 N^2 개에 가깝게 줄어듭니다.
 - 이 후보들 중 BLP(Binary Linear Programming)을 통해 실행할 그룹을 결정합니다.

Global Optimization

Taming the Search Space

Convex subgraph의 정의

그래프 $G = (V, E)$ 에 대해, 노드 집합 $V' \subset V$ 는 모든 노드 쌍 $v_1, v_2 \in V'$ 에 대해 $(v_1, u), (u, v_2) \in E$ 인 $u \in V \setminus V'$ 이 존재하지 않을 때 G 의 convex subgraph를 형성합니다.

Local Optimization

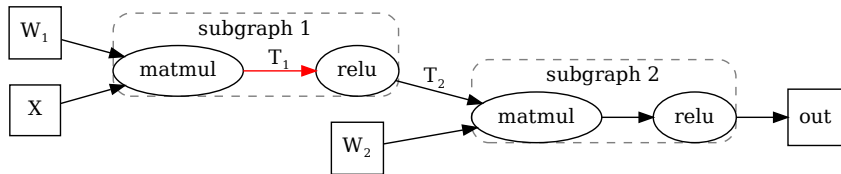
Choosing Retained Tensors

- 그룹의 텐서를 느린 메모리를 거치지 않도록 지정할 수 있습니다.
 - 빠른 메모리 공간을 차지하기 때문에, 꼭 필요한 텐서만을 선택해야 합니다.
- Operator fusion과 같이, 무식하게 모든 경우를 탐색할 수는 없습니다.

Local Optimization

Choosing Retained Tensors

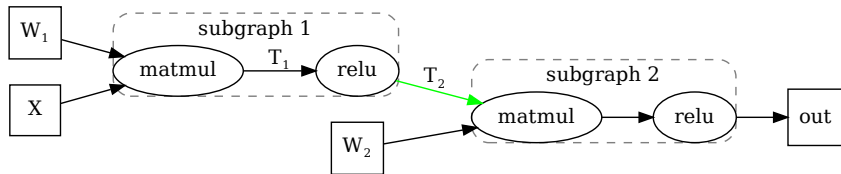
- 그룹의 텐서를 느린 메모리를 거치지 않도록 지정할 수 있습니다.
 - 빠른 메모리 공간을 차지하기 때문에, 꼭 필요한 텐서만을 선택해야 합니다.
- Operator fusion과 같이, 무식하게 모든 경우를 탐색할 수는 없습니다.



Local Optimization

Choosing Retained Tensors

- 그룹의 텐서를 느린 메모리를 거치지 않도록 지정할 수 있습니다.
 - 빠른 메모리 공간을 차지하기 때문에, 꼭 필요한 텐서만을 선택해야 합니다.
- Operator fusion과 같이, 무식하게 모든 경우를 탐색할 수는 없습니다.



제약 조건

빠른 메모리에 유지할 텐서는 그룹의 경계에 있어야 의미가 있습니다.

Local Optimization

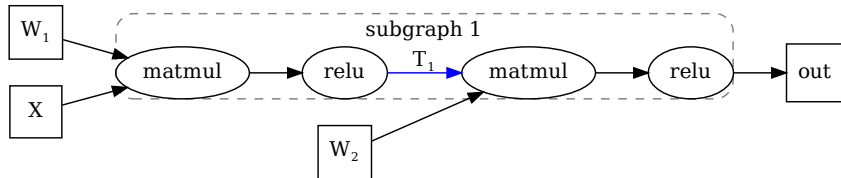
Partitioning Graphs

- 추가적으로, 유지되는 텐서는 최대 한 개의 그룹에서만 사용 가능합니다.
- 이 관점에서, 유지되는 텐서를 선택하는 것은 선택된 그룹의 분할 문제와 같습니다.

Local Optimization

Partitioning Graphs

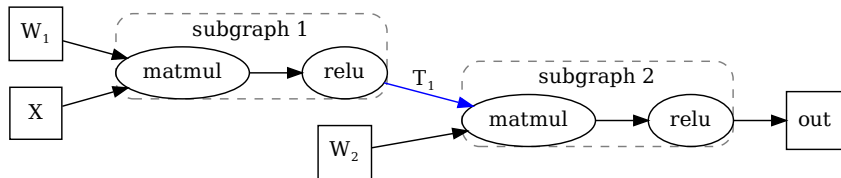
- 추가적으로, 유지되는 텐서는 최대 한 개의 그룹에서만 사용 가능합니다.
- 이 관점에서, 유지되는 텐서를 선택하는 것은 선택된 그룹의 분할 문제와 같습니다.



Local Optimization

Partitioning Graphs

- 추가적으로, 유지되는 텐서는 최대 한 개의 그룹에서만 사용 가능합니다.
- 이 관점에서, 유지되는 텐서를 선택하는 것은 선택된 그룹의 분할 문제와 같습니다.



- 실행 그룹의 각 텐서들에 대해 유지 여부를 결정해 추가적인 최적화를 구현했습니다.

Outline

TL;DR

Background

- Systems View on Deep Learning Models
- Computation Graphs
- Hardware for Deep Learning
- Operator Fusion

Solving the Problem

- Problem Analysis
- Global Optimization
- Local Optimization

Meta-strategy

Meta-strategy

군대에서 대회하기

- 가장 큰 어려움: 시간이 매우 부족합니다.
 - 파견 전이었기 때문에 물리적으로 키보드를 잡을 시간이 절대적으로 없습니다.
 - 불행 중 다행히 남는 일과시간을 논문을 읽고 설계하는데 쓸 수 있었습니다.
 - 사지방에서 논문 프린트 → 남는 일과시간에 읽기, 설계 → 사지방에서 구현

Meta-strategy

군대에서 대회하기

- 가장 큰 어려움: 시간이 매우 부족합니다.
 - 파견 전이었기 때문에 물리적으로 키보드를 잡을 시간이 절대적으로 없습니다.
 - 불행 중 다행히 남는 일과시간을 논문을 읽고 설계하는데 쓸 수 있었습니다.
 - 사지방에서 논문 프린트 → 남는 일과시간에 읽기, 설계 → 사지방에서 구현
- LLM은 확실히 Rust를 짤 때 오류가 적습니다.
 - 순수 재미를 위해 Rust를 구현 언어로 선택했지만, 의외로 도움이 되었습니다.
 - 의도가 코드에 명확하게 나타나고, 성능이 보장되면서 컴파일러가 오류를 잡아줍니다.
 - LLM의 약점인 논리적 사고를 컴파일러가 대신 해 줄 수 있습니다.

Meta-strategy

군대에서 대회하기

- 가장 큰 어려움: 시간이 매우 부족합니다.
 - 파견 전이었기 때문에 물리적으로 키보드를 잡을 시간이 절대적으로 없습니다.
 - 불행 중 다행히 남는 일과시간을 논문을 읽고 설계하는데 쓸 수 있었습니다.
 - 사지방에서 논문 프린트 → 남는 일과시간에 읽기, 설계 → 사지방에서 구현
- LLM은 확실히 Rust를 짤 때 오류가 적습니다.
 - 순수 재미를 위해 Rust를 구현 언어로 선택했지만, 의외로 도움이 되었습니다.
 - 의도가 코드에 명확하게 나타나고, 성능이 보장되면서 컴파일러가 오류를 잡아줍니다.
 - LLM의 약점인 논리적 사고를 컴파일러가 대신 해 줄 수 있습니다.
- 아 이거 좋을것같은데 → 누군가 이미 했습니다.
 - 특히 operator fusion 같은 유명한 주제는 많은 연구가 진행되어 있습니다.
 - 선행 연구를 찾다가 아이디어를 얻을 때가 많이 있었습니다.
 - 하지만 논문도 오류가 있을 수 있기 때문에 구현할 때 유의해야 합니다.

Thank You!

코드와 발표 자료 등은 깃헙 저장소에서 확인할 수 있습니다.

<https://github.com/sohnryang/mlsys2026-graph-scheduling>